# TinyML IoT Demo
# Technical and Business Report

Voice recognition personal contribution and group integration report

**Project scope**

Arduino Nano 33 BLE Sense firmware runs three TinyML models for voice, colour authentication, and movement recognition. A Python bridge forwards serial JSON to a Next.js dashboard with Postgres-backed session recording, Docker deployment, and STRIDE-informed API security.

| Field | Value |
|---|---|
| Student | Sergey Sotskiy |
| Module | Internet of Things / Cloud Computing coursework |
| Report split | Personal section: voice recognition TinyML. Group section: three-model integration, bridge, dashboard, Docker, and security. |
| Date | 29 April 2026 |

# Executive summary

This report describes a local-first TinyML Internet of Things demo that combines edge inference, cloud-style persistence, and a web dashboard into one demonstrable workflow. The system uses an Arduino Nano 33 BLE Sense class board to detect the spoken keyword start, authenticate a green colour target, and then classify board movement. The board does not connect to the network directly. Instead, it prints structured JSON over USB serial at 115200 baud and a Python bridge forwards each valid event to a Next.js API. The API stores demo sessions in PostgreSQL when Docker is used, while the dashboard shows the latest event, recorded sessions, authentication progress, and a movement replay chart.

The business value is a low-cost, explainable proof of concept for edge sensing workflows. Voice and colour gates make the system behave like an authenticated interaction rather than a raw sensor logger, while movement recognition produces useful session evidence after the user has passed both gates. Running inference on-device reduces dependency on cloud audio processing and keeps the web layer focused on persistence, visibility, and control. Docker Compose makes the dashboard and database repeatable on a server, supporting the cloud-computing requirement without forcing the microcontroller to manage Wi-Fi or database credentials.

My personal contribution was the voice recognition TinyML component. I trained and validated a keyword model in Edge Impulse, assessed its validation and on-device performance, and integrated it into the firmware state machine so a confident start detection advances the system from WAITING_FOR_VOICE to WAITING_FOR_COLOUR. The group contribution merged three Edge Impulse models into one firmware build, built the serial-to-HTTP bridge, implemented session storage and replay, added Docker deployment, and applied a STRIDE-based security layer with bearer-token protected ingest/control APIs and HMAC-signed dashboard sessions.

# 1. Introduction and business value

Many IoT demonstrations stop at collecting raw sensor values. This project takes a more product-like approach: the device must first hear a voice command, then authenticate a colour, and only then record movement. The result is a staged interaction that can be explained to non-technical stakeholders and tested live. It demonstrates how embedded machine learning can add local intelligence to a small sensor platform while a web application provides the visibility and evidence trail expected from a connected system.

The Arduino Nano 33 BLE Sense is suitable for this because it combines a microphone, an APDS9960 colour/proximity sensor, and an LSM9DS1 inertial measurement unit with a microcontroller capable of TinyML workloads (Arduino, n.d.-a). Edge Impulse supports training and deploying embedded models as Arduino libraries, making it appropriate for a coursework prototype where the model pipeline and firmware deployment both need to be visible (Edge Impulse, n.d.-a). The system also reflects a practical deployment constraint: the board performs local sensing and inference, but a laptop bridge handles networking and the cloud-facing dashboard.

The business value is not only technical novelty. A staged edge workflow can be adapted to access control, equipment-use logging, gesture-controlled interfaces, or training demonstrations where the system must prove that a user completed a required sequence. The dashboard records sessions, stores movement samples, and can stop a run from the browser. This creates auditable evidence for a demo, while Docker and PostgreSQL make the server side repeatable and persistent rather than a temporary local page.

# 2. Personal contribution: voice recognition TinyML

My personal contribution was the voice recognition stage. The purpose of this model was deliberately narrow: identify the keyword start strongly enough to trigger the next phase, while treating background speech or silence as unknown. This small scope made the model suitable for a constrained embedded device because it avoided open-ended speech recognition and instead used keyword spotting. In the complete system, this model is the first authentication gate, so its output controls whether the demo remains in WAITING_FOR_VOICE or advances to colour authentication.

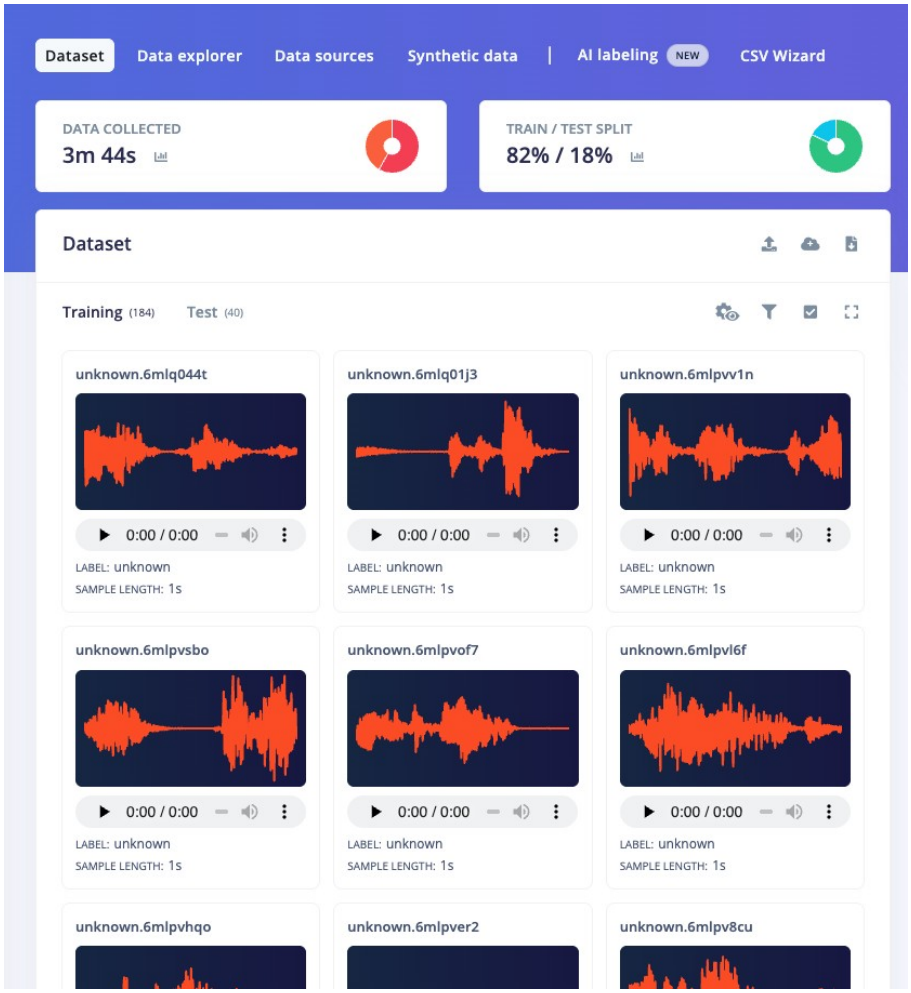


*Fig. P1. Edge Impulse voice dataset showing the start and unknown classes used to train the keyword model.*

The dataset was organised in Edge Impulse around the target command and a negative class. Figure P1 shows the captured voice data used for this model. The positive class represents the word start, while the unknown class gives the classifier examples of audio that should not trigger the state change. This distinction is important because a demo environment can contain speech, movement noise, keyboard sounds, and room noise. A binary keyword model is only useful if it learns both the intended command and the non-command background distribution.

*Fig. P2. Data explorer view used to inspect recorded voice samples before training.*

Before training, the data explorer was used to inspect whether the recorded samples looked plausible and whether the classes were separated enough for a simple audio classifier. Edge Impulse audio workflows normally transform raw audio into signal-processing features before classification, rather than feeding a full waveform directly to an embedded neural network (Edge Impulse, n.d.-b). This approach is appropriate for microcontrollers because it reduces the amount of data passed into the model and makes inference more predictable on constrained hardware. The model therefore fits the TinyML pattern: collect representative data, extract compact features, train a classifier, and deploy the inference code to the target.

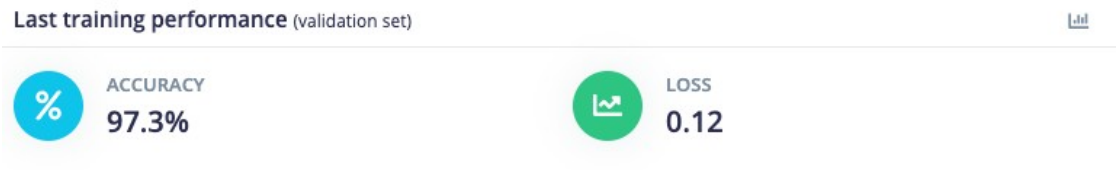


*Fig. P3. Final validation performance after training the voice model.*

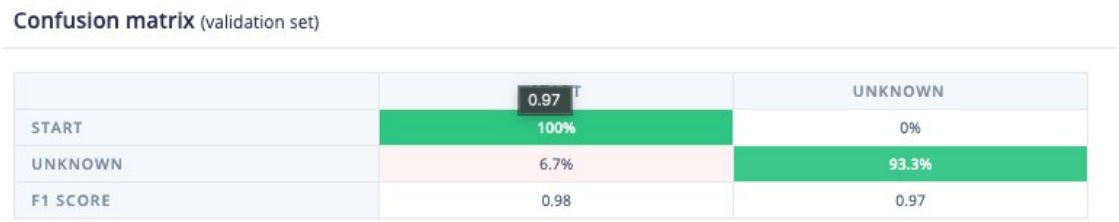


*Fig. P4. Confusion matrix showing how validation samples were classified.*

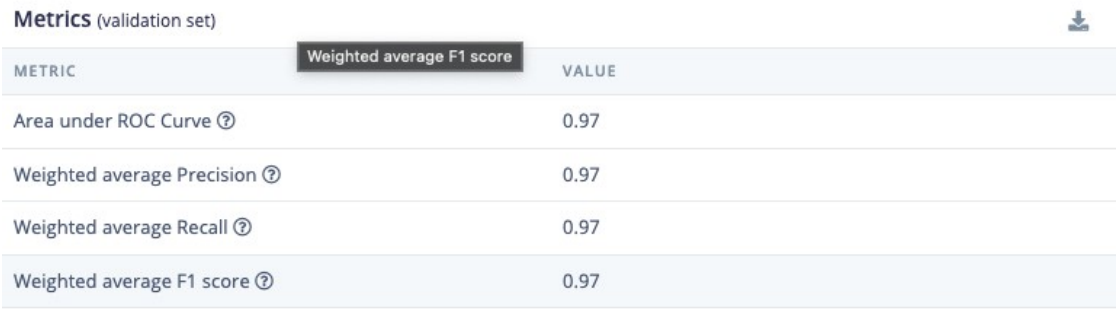


*Fig. P5. Validation metrics used to judge voice model quality.*

The validation results in Figures P3, P4, and P5 were used to decide whether the model was reliable enough for the live state machine. The confusion matrix is particularly useful because the cost of a false positive is high: the system would skip the voice phase and immediately ask for colour authentication. For a demo, occasional false negatives are less damaging because the user can say start again, but false positives make the interaction look uncontrolled. This is why the firmware also includes state-machine protections, including an arming delay after boot and a confidence threshold before the start event is accepted.

*Fig. P6. On-device performance estimate for memory and latency suitability.*

On-device performance was checked before integration. Figure P6 shows the deployment estimate used to judge whether the voice model could run within the board's memory and timing constraints. This matters because the final firmware also includes a colour model, a movement model, sensor libraries, the Edge Impulse SDK, and serial JSON output. TinyML deployments need to fit within flash and RAM and must complete inference quickly enough that the user experience still feels live (Google, n.d.). The voice model was therefore evaluated not only by accuracy but also by whether it could run continuously on the board.

```
[bridge] POST movement -> 200 {"event":"movement","ax":0.054,"ay":0.104,"az":0.979,"gx":3.479,"gy":0.366,"gz":0.854,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.055,"ay":0.103,"az":0.979,"gx":3.479,"gy":0.183,"gz":0.854,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.055,"ay":0.103,"az":0.979,"gx":3.479,"gy":0.305,"gz":0.854,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.055,"ay":0.103,"az":0.979,"gx":3.357,"gy":0.244,"gz":0.854,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.054,"ay":0.103,"az":0.979,"gx":3.357,"gy":0.305,"gz":0.916,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.054,"ay":0.102,"az":0.978,"gx":3.418,"gy":0.061,"gz":0.977,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.053,"ay":0.102,"az":0.979,"gx":3.357,"gy":0.427,"gz":0.916,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.054,"ay":0.104,"az":0.979,"gx":3.479,"gy":0.366,"gz":0.854,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.053,"ay":0.103,"az":0.979,"gx":3.479,"gy":0.366,"gz":0.854,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.055,"ay":0.105,"az":0.978,"gx":3.601,"gy":0.183,"gz":0.916,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":0.058,"ay":0.106,"az":0.978,"gx":3.357,"gy":0.183,"gz":0.977,"movementClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":t
rue,"direction":"down"}
```

*Fig. P7. Runtime bridge output proving the deployed voice model emits voice_debug and voice_start.*

After deployment, the firmware emitted voice_debug events containing the current state, threshold, arming status, streak, and classification scores. Figure P7 shows this runtime evidence. The implemented threshold was 0.75, with a startup arming delay to reduce immediate false triggers. For demo reliability the required streak was reduced to one confident detection, because earlier tests showed that demanding multiple consecutive high-confidence windows made the system miss spoken commands. This is a practical trade-off: the validation model looked capable, but a live room, microphone placement, and speech variation can still reduce confidence. The final design keeps the threshold high, adds an arming delay, and exposes debug scores so the operator can understand why the board did or did not advance.

The main limitation of my voice component is that it is trained for a small vocabulary and a limited set of recording conditions. It should not be presented as general speech recognition. Its value is that it provides a focused, explainable trigger for a staged IoT workflow. Future improvements would include collecting more speakers, adding more negative samples from the actual demo room, testing different thresholds, and disabling debug output once the live behaviour is stable.

# 3. Group system architecture

The full group system is split into firmware, bridge, and web/cloud layers. Figure G1 shows the data flow and trust boundaries. The Arduino layer owns sensing and inference. It samples the microphone through the PDM library, reads colour data from the APDS9960 sensor, and reads acceleration and gyroscope data from the LSM9DS1 IMU. The firmware uses these inputs to run three TinyML models and prints JSON events to USB serial. The board intentionally does not connect to the network, which keeps the microcontroller firmware simpler and avoids putting server credentials on the device.

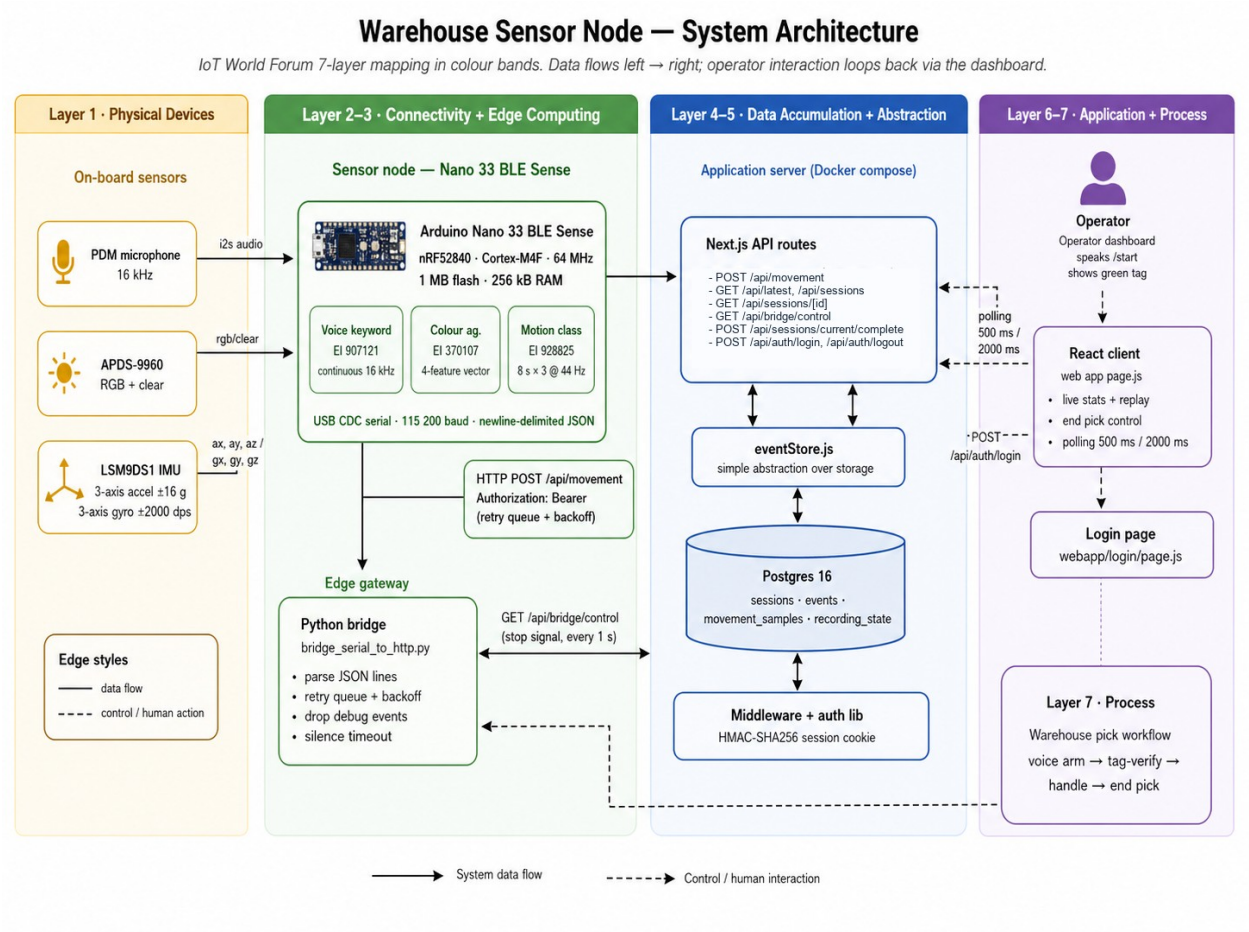


*Fig. G1. End-to-end architecture showing Arduino sensing, the serial bridge, web API routes, Postgres storage, dashboard auth, and the operator workflow.*

The bridge layer runs on the laptop connected to the board. It opens the serial port at 115200 baud, ignores non-JSON lines, validates that each line is a JSON object, and posts valid payloads to POST /api/movement with BRIDGE_API_TOKEN on deployed runs. It also polls GET /api/bridge/control so the dashboard can request a clean bridge shutdown. The bridge has a bounded retry queue with backoff for transient HTTP failures, a silence timeout for dead serial streams, and a serial reconnect retry path for USB reset or macOS device reconfiguration errors. This design was chosen because serial is reliable for the Nano board during a demo, while Python and HTTP are easier to debug than adding networking directly to the firmware.

The web layer is a Next.js application. Route handlers implement ingestion, latest-event, session-list, session-detail, complete-session, bridge-control, login, and logout APIs. Next.js route handlers are designed for custom request handlers in the app directory (Next.js, n.d.). When DASHBOARD_PASSWORD is configured, middleware gates the dashboard and read APIs behind an HMAC-signed session cookie issued by POST /api/auth/login. When DATABASE_URL is configured, the event store persists data in PostgreSQL; without it, the same code falls back to memory for fast local testing. PostgreSQL was selected because session events and movement samples naturally fit relational tables with timestamps and foreign keys (PostgreSQL Global Development Group, n.d.).

# 4. Firmware and three-model integration

The firmware state machine has three main states: WAITING_FOR_VOICE, WAITING_FOR_COLOUR, and TRACKING_MOVEMENT. It starts in the voice state after reset. A successful start classification emits voice_start and moves to colour. A successful green classification emits colour_authenticated and moves to movement tracking. Once movement tracking starts, the board continuously sends movement events with raw IMU values and the movement model's predicted class and confidence.

```
(.venv) (base) MacBook-Air-6:Iot sergeysotskiy$ python bridge/serial_to_http.py \
>   --port /dev/cu.usbmodem1201 \
>   --endpoint http://localhost:3001/api/movement \
>   --api-token "$BRIDGE_API_TOKEN" \
>   --debug-downsample 1 \
>   --verbose
[bridge] polling control endpoint http://localhost:3001/api/bridge/control
[bridge] opening /dev/cu.usbmodem1201 @ 115200, forwarding to http://localhost:3001/api/movement
[bridge] POST voice_debug -> 200 {"event":"voice_debug","state":"WAITING_FOR_VOICE","threshold":0.75,"armed":false,"streak":0,
"requiredStreak":1,"latched":false,"scores":{"start":0.20703,"unknown":0.79297}}
[bridge] POST voice_debug -> 200 {"event":"voice_debug","state":"WAITING_FOR_VOICE","threshold":0.75,"armed":false,"streak":0,
"requiredStreak":1,"latched":false,"scores":{"start":0.38281,"unknown":0.61719}}
[bridge] POST voice_debug -> 200 {"event":"voice_debug","state":"WAITING_FOR_VOICE","threshold":0.75,"armed":false,"streak":0,
"requiredStreak":1,"latched":false,"scores":{"start":0.19531,"unknown":0.80469}}
[bridge] POST voice_debug -> 200 {"event":"voice_debug","state":"WAITING_FOR_VOICE","threshold":0.75,"armed":true,"streak":0,"
requiredStreak":1,"latched":false,"scores":{"start":0.28125,"unknown":0.71875}}
[bridge] POST voice_debug -> 200 {"event":"voice_debug","state":"WAITING_FOR_VOICE","threshold":0.75,"armed":true,"streak":0,"
requiredStreak":1,"latched":false,"scores":{"start":0.05859,"unknown":0.94141}}
[bridge] POST voice_start -> 200 {"event":"voice_start"}
[bridge] POST colour_debug -> 200 {"event":"colour_debug","state":"WAITING_FOR_COLOUR","r":1,"g":1,"b":1,"clear":2,"greenNow":
```

*Fig. G2. Bridge output showing setup status, voice debug, and the start transition into colour authentication.*

Integrating three Edge Impulse exports was the hardest firmware issue. Each generated Arduino library normally expects to be the only Edge Impulse model in the sketch, so including three generated inferencing headers directly creates conflicting symbols such as run_classifier, run_classifier_init, model variables, and preprocessor macros. The solution was to create a local combined_inferencing Arduino library. It contains one SDK copy, merged model metadata, and explicit handles for the three models: voice model 970121, colour model 970107, and movement model 928825. The sketch includes only combined_inferencing.h and calls classifier functions through the selected handle.

The colour model replaced the earlier scripted colour heuristic. The firmware reads APDS9960 red, green, blue, and clear values and maps them to the model input order ch1, ch2, ch3, and ch4. The model labels are blue, green, other, and red. The firmware requires the green class to exceed a 0.70 threshold for three consecutive reads before colour authentication succeeds. This is more robust than a single raw RGB threshold because the model can learn the teammate's recorded colour examples rather than relying only on a fixed ratio.

```
[bridge] POST colour_debug -> 200 {"event":"colour_debug","state":"WAITING_FOR_COLOUR","r":1,"g":1,"b":1,"clear":2,"greenNow":
false,"streak":0,"greenConfidence":0.5,"topLabel":"green","topConfidence":0.5,"threshold":0.7,"scores":{"blue":0.21094,"green"
:0.5,"other":0.05469,"red":0.23437}}
[bridge] POST colour_debug -> 200 {"event":"colour_debug","state":"WAITING_FOR_COLOUR","r":1,"g":1,"b":1,"clear":2,"greenNow":
false,"streak":0,"greenConfidence":0.5,"topLabel":"green","topConfidence":0.5,"threshold":0.7,"scores":{"blue":0.21094,"green"
:0.5,"other":0.05469,"red":0.23437}}
[bridge] POST colour_debug -> 200 {"event":"colour_debug","state":"WAITING_FOR_COLOUR","r":1,"g":1,"b":1,"clear":2,"greenNow":
false,"streak":0,"greenConfidence":0.5,"topLabel":"green","topConfidence":0.5,"threshold":0.7,"scores":{"blue":0.21094,"green"
:0.5,"other":0.05469,"red":0.23437}}
[bridge] POST colour_debug -> 200 {"event":"colour_debug","state":"WAITING_FOR_COLOUR","r":1,"g":1,"b":1,"clear":2,"greenNow":
false,"streak":0,"greenConfidence":0.5,"topLabel":"green","topConfidence":0.5,"threshold":0.7,"scores":{"blue":0.21094,"green"
:0.5,"other":0.05469,"red":0.23437}}
[bridge] POST colour_debug -> 200 {"event":"colour_debug","state":"WAITING_FOR_COLOUR","r":4,"g":3,"b":3,"clear":9,"greenNow":
false,"streak":0,"greenConfidence":0.55078,"topLabel":"green","topConfidence":0.55078,"threshold":0.7,"scores":{"blue":0.20703
,"green":0.55078,"other":0.05469,"red":0.18359}}
[bridge] POST colour_authenticated -> 200 {"event":"colour_authenticated","colour":"green"}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.014,"ay":-0.0,"az":0.988,"gx":2.869,"gy":0.183,"gz":0.793,"movementC
```

*Fig. G3. Bridge output showing colour debug, green tag authentication, and transition into movement tracking.*

The movement model uses 88 accelerometer samples sampled at 44 Hz, with acceleration converted from g to m/s^2 before inference. The labels are down, idle, left, right, and up. The firmware maintains a rolling buffer, runs the model after the buffer is full, and emits movement JSON about every 250 ms. It includes both raw accelerometer and gyroscope values because raw telemetry is useful when debugging model mistakes, while movementClass and movementConfidence provide the TinyML interpretation.

```
[bridge] POST movement -> 200 {"event":"movement","ax":-0.013,"ay":-0.001,"az":0.988,"gx":2.869,"gy":0.244,"gz":0.854,"movemen
tClass":"collecting","movementConfidence":0.0,"movementBufferReady":false,"movementClassifierOk":false}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.013,"ay":-0.001,"az":0.988,"gx":2.747,"gy":0.244,"gz":0.916,"movemen
tClass":"collecting","movementConfidence":0.0,"movementBufferReady":false,"movementClassifierOk":false}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.013,"ay":-0.001,"az":0.989,"gx":3.052,"gy":0.183,"gz":0.793,"movemen
tClass":"collecting","movementConfidence":0.0,"movementBufferReady":false,"movementClassifierOk":false}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.015,"ay":-0.001,"az":0.988,"gx":2.747,"gy":0.0,"gz":0.977,"movementC
lass":"collecting","movementConfidence":0.0,"movementBufferReady":false,"movementClassifierOk":false}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.014,"ay":-0.001,"az":0.988,"gx":2.93,"gy":0.305,"gz":0.916,"movement
Class":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":true,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.014,"ay":-0.001,"az":0.989,"gx":2.808,"gy":0.366,"gz":1.038,"movemen
tClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":true,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.014,"ay":-0.001,"az":0.99,"gx":2.869,"gy":0.061,"gz":0.916,"movement
Class":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":true,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.014,"ay":-0.002,"az":0.989,"gx":2.991,"gy":0.183,"gz":0.793,"movemen
tClass":"down","movementConfidence":0.99609,"movementBufferReady":true,"movementClassifierOk":true,"direction":"down"}
[bridge] POST movement -> 200 {"event":"movement","ax":-0.011,"ay":0.002,"az":0.99,"gx":2.625,"gy":0.061,"gz":0.793,"movementC
```

*Fig. G4. Bridge output showing movement collection and classified handling direction events.*

This firmware design keeps the demo explainable. Each state has one responsibility, and each state emits debug evidence while the demo is being stabilised. The trade-off is that the board state is one-way. Restarting the bridge does not reset the board, so a new full demo requires a board reset to return to WAITING_FOR_VOICE. This was accepted because it keeps the firmware logic simple and deterministic for a live demonstration.

## 5. Bridge, dashboard, and data recording

The Python bridge is the boundary between embedded serial output and the web system. It uses pyserial to read the board, parses only lines that begin with JSON object syntax, and uses the requests library to POST each event. Verbose mode prints the HTTP status and compact JSON payload, which made it possible to debug the firmware sequence without opening the Arduino Serial Monitor. To keep normal sessions readable, voice_debug and colour_debug are dropped by default unless --debug-downsample is set. Only one process can own the serial port at a time, so the bridge and Serial Monitor are not run together.

The dashboard is built around session recording rather than only the latest event. A setup_status event starts a new session. Later voice_start, colour_authenticated, and movement events attach to the active session. The database stores sessions, raw events, movement samples, and recording state. The movement_samples table separates chartable numeric telemetry from the raw JSON payload, while the events table preserves the original event for auditability. This gives the project both a live monitor and a persistent evidence store.

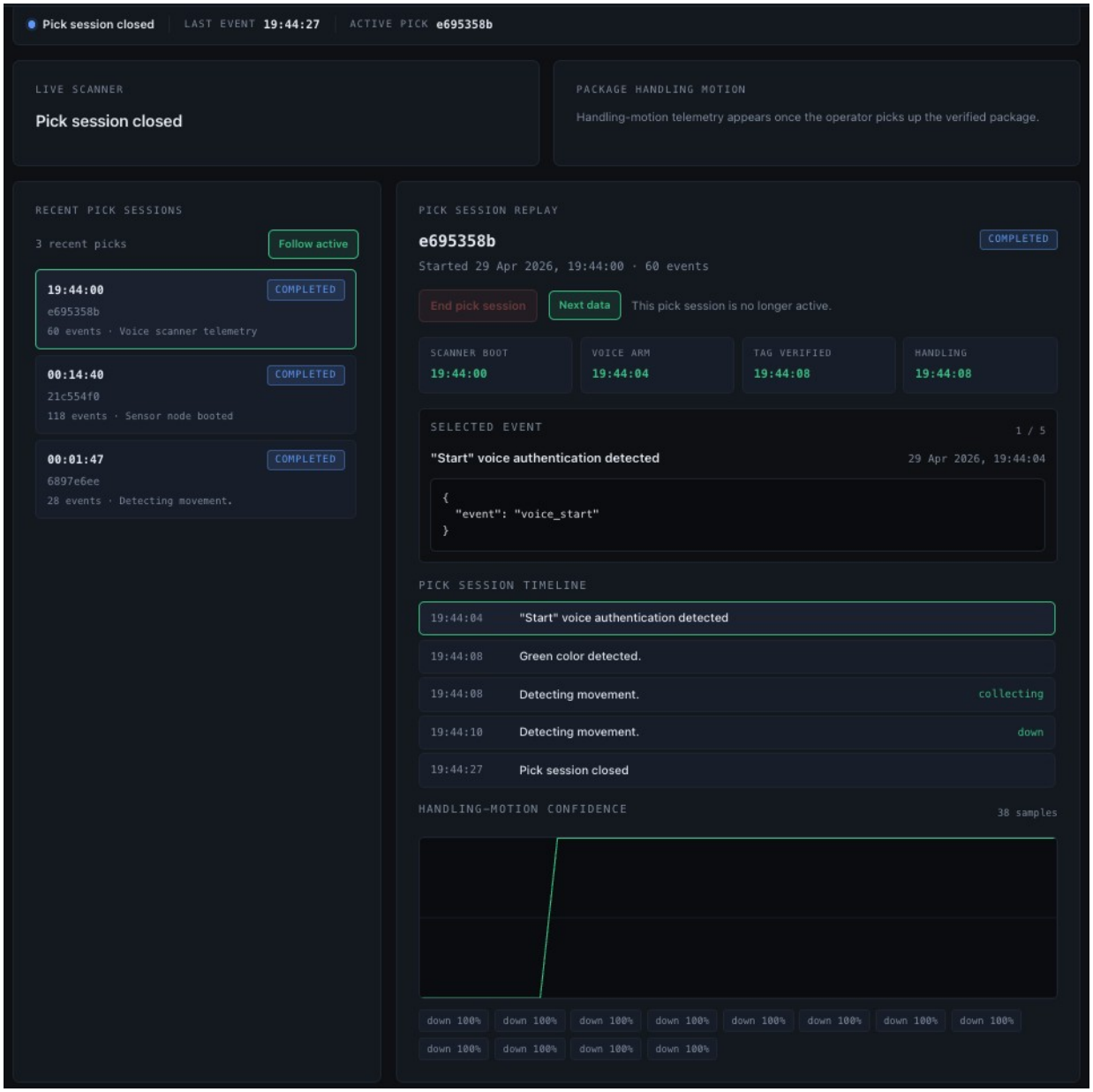


*Fig. G5. Dashboard replay showing a completed pick session with voice authentication, green colour detection, movement events, and recorded-session history.*

The dashboard shows business-facing status rather than only developer logs. It reports whether the voice, colour, and movement stages are waiting or complete, displays the raw latest JSON, and lists recorded sessions. The current operator-facing labels are '"Start" voice authentication detected', 'Green color detected.', and 'Detecting movement.' The End pick session button controls the lifecycle from the web UI. When the stop action is triggered, the server accepts either the logged-in operator's session cookie or ADMIN_API_TOKEN for external automation. If voice or colour authentication was not completed, the incomplete data is deleted because it does not represent a successful demo session. Stop also sets bridge-control state so the bridge exits cleanly instead of continuing to post movement samples.

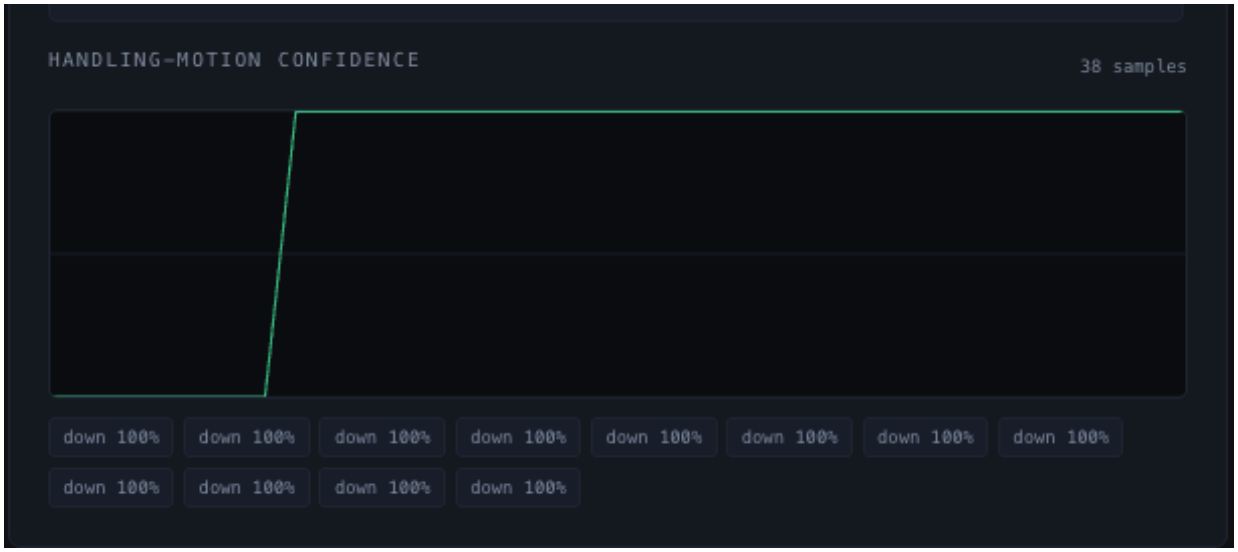


*Fig. G6. Recorded movement confidence visualised in the web dashboard.*

Figure G6 shows why recording sessions is more valuable than printing logs. The dashboard turns repeated movement events into a replayable chart with confidence values and timestamps. This supports post-demo analysis: the group can show that the system detected voice, authenticated colour, entered tracking, and captured movement data. For a coursework demo, this is important because it provides evidence even after the live interaction has ended.

# 6. Cloud deployment and Docker

The deployable server path uses Docker Compose to run the web application and PostgreSQL database. Compose is designed for defining and running multi-container applications with a single configuration file (Docker, n.d.-a). In this project, the postgres service uses the postgres:16-alpine image and a named volume for persistent data, while the web service builds the Next.js application from web/Dockerfile. The web container runs the migration script before starting the production server, so a fresh server can create the required tables automatically.

The server deployment intentionally excludes the Arduino and bridge from Docker. The bridge needs direct USB serial access to the board, so it stays on the laptop connected to the device and posts to the server URL. This separation is practical: the cloud/server part stores and visualises data, while the local laptop manages physical hardware. Docker Compose environment variables configure Postgres credentials, the public web port, and required API tokens (Docker, n.d.-b). The deployment script _update_server runs git pull --ff-only, rebuilds the containers, restarts the stack, and prints the service state.

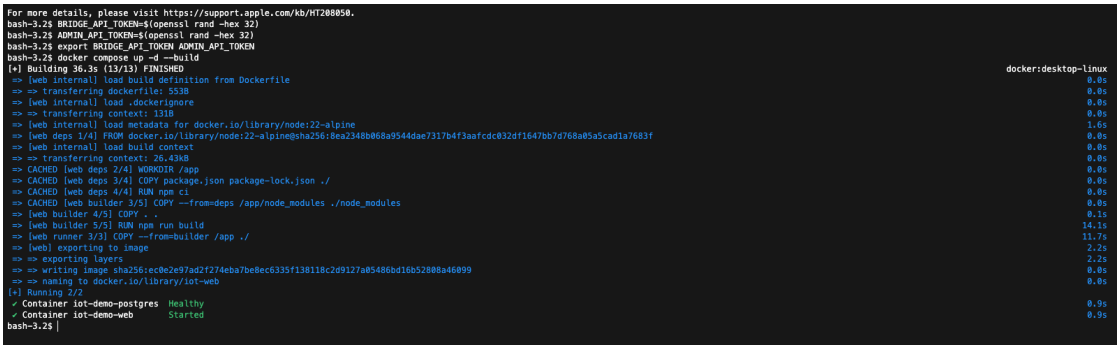


*Fig. G7. Docker Compose stack running the web application and database services.*

Postgres is not exposed broadly to the network. The Compose file binds the database port to 127.0.0.1 on the host, while the web container connects to it through the internal Docker network. This provides enough persistence and cloud-style deployment evidence for the project without turning the database into a public service. The default credentials are suitable only for demonstration and can be changed in .env before deployment.

# 7. Security using STRIDE

The project uses STRIDE as a lightweight security protocol and threat modelling method. STRIDE classifies threats as spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege (Microsoft, n.d.). For this system, the main assets are sensor events, session history, control actions, and shared secrets. The main trust boundaries are USB serial from the board to the bridge, HTTP from bridge/browser to the web API, and web API to PostgreSQL inside Docker.

The implemented mitigation separates bridge credentials from operator credentials. POST /api/movement and GET /api/bridge/control require BRIDGE_API_TOKEN when configured. POST /api/sessions/current/complete accepts either the logged-in operator's HMAC-signed session cookie or ADMIN_API_TOKEN for external scripts. When DASHBOARD_PASSWORD and SESSION_SECRET are configured, the dashboard and read APIs require the operator session cookie. Bearer tokens are sent in the Authorization header, following the standard bearer-token pattern defined by RFC 6750 (IETF, 2012). On the server, token and password comparison uses Node.js crypto.timingSafeEqual to avoid simple timing leaks in equality checks (Node.js, n.d.).

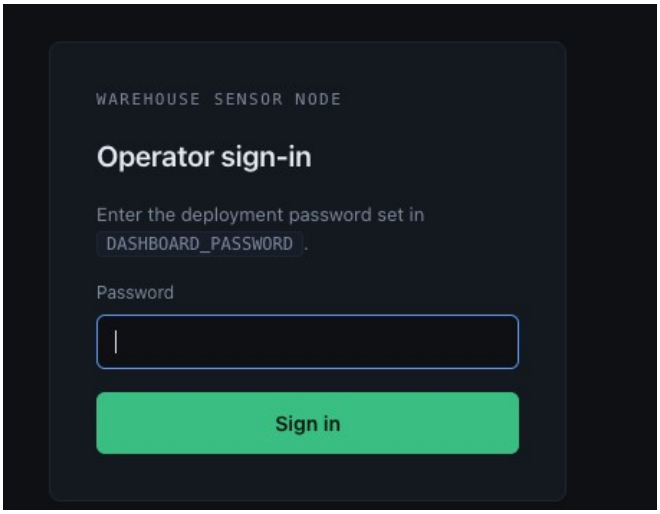


*Fig. G8. Operator sign-in page used to issue the dashboard session cookie.*

```
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
bash-3.2$ curl -i -X POST http://127.0.0.1:3000/api/movement \
>   -H 'Content-Type: application/json' \
>   -d '{"event":"setup_status","state":"WAITING_FOR_VOICE"}'
HTTP/1.1 401 Unauthorized
vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
content-type: application/json
Date: Tue, 28 Apr 2026 21:14:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked
```

*Fig. G9. Unauthorized API request returning HTTP 401 when no bearer token is supplied.*

```
bash-3.2$ curl -i -X POST http://127.0.0.1:3000/api/movement \
>   -H 'Content-Type: application/json' \
>   -H 'Authorization: Bearer test-bridge-token' \
>   -d '{"event":"setup_status","state":"WAITING_FOR_VOICE"}'
HTTP/1.1 200 OK
vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
content-type: application/json
Date: Tue, 28 Apr 2026 21:15:42 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"ok":true,"latest":{"event":"setup_status","state":"WAITING_FOR_VOICE","sessionId":"69a8ef24-121d-4eab-b2b2-01ca35f690c4","receivedAt":"2026-04-28T21:15:42.732Z"},"session":{"id":"69a8ef24-121d-4eab-b2b2-01ca35f690c4",
"startedAt":"2026-04-28T21:15:42.732Z","endedAt":null,"status":"active","triggerEvent":"setup_status","lastEventAt":"2026-04-28T21:15:42.732Z","eventCount":1,"voiceStartedAt":null,"colourAuthenticatedAt":null,"movementS
tartedAt":null,"metadata":{"initialState":"WAITING_FOR_VOICE","source":"memory_fallback"}}}bash-3.2$
```

*Fig. G10. Authorized API request succeeding with a valid bearer token.*

Figures G8, G9, and G10 show the two authentication surfaces. The goal is not enterprise identity management, but practical protection for a demo server. A random network user should not be able to fabricate board events, stop the bridge,

view dashboard data when the password gate is enabled, or complete/delete sessions. The browser path uses the dashboard password and a signed session cookie, so no admin bearer token is stored in localStorage.

Residual risks remain. If the server is exposed directly over plain HTTP, bearer tokens and login traffic can be observed on the network; a real public deployment should use HTTPS behind a reverse proxy. Tokens and the dashboard password are shared secrets rather than named user accounts, so anyone with the relevant secret has that role. Physical access to the board and bridge laptop is still trusted. These residual risks are acceptable for a university demonstration but should be addressed before a production deployment.

# 8. Testing and evaluation

The manual test procedure starts the web application, signs in to the dashboard when DASHBOARD_PASSWORD is configured, starts the Python bridge with BRIDGE_API_TOKEN, resets the board, and observes setup_status. The operator says start, shows green to the APDS9960 sensor, then moves the board. A successful run should record voice_start, colour_authenticated, and movement events in that order. The screenshots in this report provide evidence for each phase: voice transition in Figure G2, colour authentication in Figure G3, movement classification in Figure G4, session replay in Figure G5, and movement confidence replay in Figure G6.

API tests were also used. Unauthorised mutation requests return 401 when tokens are configured, while authorised requests succeed. Dashboard auth tests check that unauthenticated users are redirected to login, invalid passwords fail, and valid passwords issue the session cookie used by the operator UI. Docker testing checks that Compose starts both services and that migrations complete before the web server runs. The main evaluation result is that the system behaves as a full pipeline: local TinyML inference controls state transitions, the bridge reliably forwards JSON, the web app stores and visualises sessions, and security controls prevent unauthorised access on the deployed path.

# 9. Conclusion

The project demonstrates a complete TinyML IoT workflow rather than a disconnected model or a raw sensor dashboard. My voice recognition model provides the first interaction gate, and the group integration combines voice, colour, and movement models into a single firmware state machine. The Python bridge, Next.js dashboard, PostgreSQL storage, Docker deployment, bearer-token bridge security, and signed dashboard sessions turn the board output into a deployable recorded-session system.

The strongest business value is explainability. A stakeholder can watch the system progress from voice command to colour authentication to movement tracking and then review the captured session afterwards. Future work should improve the dataset size for all models, add HTTPS for public deployment, replace shared tokens with named users, and reduce debug output once the demo is stable. Even with those limitations, the current prototype shows how low-cost edge AI can be connected to a cloud-style evidence and control layer.

# References


Arduino (n.d.-a) Nano 33 BLE Sense. Available at: https://docs.arduino.cc/hardware/nano-33-ble-sense/ (Accessed: 28 April 2026).

Arduino (n.d.-b) Arduino_APDS9960 library. Available at: https://github.com/arduino-libraries/Arduino_APDS9960 (Accessed: 28 April 2026).

Arduino (n.d.-c) Arduino_LSM9DS1 library. Available at: https://github.com/arduino-libraries/Arduino_LSM9DS1 (Accessed: 28 April 2026).

Docker (n.d.-a) Docker Compose documentation. Available at: https://docs.docker.com/compose/ (Accessed: 28 April 2026).

Docker (n.d.-b) Interpolation: Docker Compose environment variables. Available at: https://docs.docker.com/compose/how-tos/environment-variables/variable-interpolation/ (Accessed: 28 April 2026).

Docker (2022) How to use the Postgres Docker Official Image. Available at: https://www.docker.com/blog/how-to-use-the-postgres-docker-official-image/ (Accessed: 28 April 2026).

Edge Impulse (n.d.-a) Run Arduino library. Available at: https://docs.edgeimpulse.com/docs/run-inference/arduino-library (Accessed: 28 April 2026).

Edge Impulse (n.d.-b) Sound recognition. Available at: https://docs.edgeimpulse.com/docs/tutorials/audio-classification (Accessed: 28 April 2026).

Google (n.d.) LiteRT for Microcontrollers. Available at: https://ai.google.dev/edge/litert/microcontrollers/overview (Accessed: 28 April 2026).

IETF (2012) RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage. Available at: https://www.rfc-editor.org/rfc/rfc6750 (Accessed: 28 April 2026).

Mermaid (n.d.) Flowcharts syntax. Available at: https://mermaid.js.org/syntax/flowchart.html (Accessed: 28 April 2026).

Microsoft (n.d.) Threats: Microsoft Threat Modeling Tool. Available at: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats (Accessed: 28 April 2026).

Next.js (n.d.) Route Handlers. Available at: https://nextjs.org/docs/app/getting-started/route-handlers-and-middleware (Accessed: 28 April 2026).

Node.js (n.d.) Crypto: crypto.timingSafeEqual. Available at: https://nodejs.org/api/crypto.html#cryptotimingsafeequala-b (Accessed: 28 April 2026).

PostgreSQL Global Development Group (n.d.) PostgreSQL documentation. Available at: https://www.postgresql.org/docs/ (Accessed: 28 April 2026).

# Appendix A. Evidence map

| Evidence | What it proves |
|---|---|
| Figures P1-P7 | Personal voice TinyML workflow, validation, on-device feasibility, and deployed runtime output. |
| Figures G1-G6 | End-to-end architecture, firmware state transitions, bridge forwarding, dashboard session recording, and movement replay. |
| Figure G7 | Docker Compose deployment path for the web app and PostgreSQL database. |
| Figures G8-G10 | Dashboard login and STRIDE-inspired API protection, with unauthorised and authorised request outcomes. |